

1. Title of the Invention

Mint-to Logic™ System for Validation-Based Digital Routing and Protocol Execution

2. Background of the Invention

Modern digital systems rely heavily on authentication and routing logic to determine user access, asset control, and protocol initiation. However, most current models lack modular validation frameworks, clean token lifecycle controls, and flexible routing strategies that can adapt to real-time contextual data or ethical AI systems.

Conventional blockchain and smart contract systems, while secure, often sacrifice adaptability, real-world triggers, and granular auditability. There exists a need for a validation-first, action-triggered system that mints unique execution tokens — known herein as Mint Units — only after confirmation, and routes them intelligently through defined protocol paths such as burn, limbo, access, or code execution.

This invention introduces a logic framework and system architecture called the Mint-to Logic system, which is designed to securely validate input, mint unique logic units, and route them through scalable, protocol-defined flows that can power everything from secure access to AI-driven tasks.

3. Summary of the Invention

The Mint-to Logic™ system is a modular protocol infrastructure that enables secure, validated digital routing through Mint Units™ — one-time logic objects that are created only after specific triggers are met. Once minted, these Mint Units are routed via embedded metadata to one of several defined execution protocols, such as burn (immediate destruction), limbo (conditional or delayed access), access (permission granting), or code (smart logic execution).

Each Mint Unit contains embedded routing instructions, timestamps, and validation metadata. The system is capable of interfacing with real-world actions, biometric inputs, environmental data, and AI systems to determine routing outcomes. This architecture provides a secure, scalable way to direct actions in both digital and physical systems — from entry tokens to AI commands — while preserving audit trails and maintaining control over each logic unit's lifecycle.

This system lays the foundation for next-generation identity management, programmable trust layers, event-based tokenization, and cross-protocol AI integration.

4. Brief Description of the Drawings

FIG. 1 is a high-level system architecture diagram of the Mint-to Logic framework, illustrating the sequential flow from source input through validation, Mint Unit generation, routing, protocol execution, and audit logging.

FIG. 2 is an expanded view of the Source Input architecture [100], depicting various input types including biometric, device-based, user data, environmental, and contextual sources.

FIG. 3 is an expanded view of the Validation / Trigger architecture [200], showing authentication layers, trigger mechanisms, and logic initiators leading to Mint Unit creation.

FIG. 4 is an expanded view of the Mint Unit Generation architecture [300], including unique identifier hashing, metadata encoding, timestamp assignment, and initial state control.

FIG. 5 is an expanded view of the Routing Decision Layer [400], illustrating the system logic that determines the protocol pathway the Mint Unit will follow based on embedded metadata and validation outcome.

FIG. 6 is an expanded view of the Mint-to-Burn Protocol architecture [500a], representing immediate-use, single-purpose tokens that are destroyed after a completed transaction or action.

FIG. 7 is an expanded view of the Mint-to-Limbo Protocol architecture [500b], illustrating conditional token holds, AI-evaluated decisions, time-based expiration, and rerouting pathways.

FIG. 8 is an expanded view of the Mint-to-Access Protocol architecture [500c], showing validation-based access control to physical, digital, or system assets, along with optional time/context restrictions and lifecycle finalization.

FIG. 9 is an expanded view of the Mint-to-Code Protocol architecture [500d], depicting logic engagement for smart contract triggers, AI instruction delivery, system automation, or external execution events.

5. Detailed Description of the Preferred Embodiments

[100] Source Input / Initiation

The system begins with a source input module [100], which serves as the primary intake point for any data or event intended to initiate a validation-triggered minting sequence. This input may originate from one or more sources including but not limited to user-submitted personal identifiers, biometric scans, device-based credentials, sensor data, time-sensitive triggers, financial information, or environmental conditions.

As illustrated in FIG. 2, the source input architecture is modular and designed to support multiple, parallel input types. For example, submodules [100.1], [100.2], and [100.3] may receive data from biometric readers, password or code entry fields, or device fingerprinting systems, respectively. Additional submodules [100.4] and [100.5] may include external API inputs, environmental signal receivers, or geolocation inputs. Optional component [100.6] may capture contextual meta-input such as time of day, network condition, or behavioral heuristics.

These submodules may function independently or in conjunction, forming a flexible, multi-channel intake layer that ensures the Mint-to Logic system is adaptable to evolving validation methods and scalable to numerous use cases. Each input type is recorded with a timestamp and routed to the validation subsystem [200] only when system readiness and intake completeness is confirmed.

[200] Validation / Trigger

The validation subsystem [200], as illustrated in FIG. 3, is responsible for evaluating incoming data from the source input layer [100] and determining whether the input satisfies pre-defined criteria necessary to initiate the minting of a Mint Unit™. This subsystem may incorporate single or multi-factor authentication mechanisms, such as biometric verification [200.1], passcode entry [200.2], or behavioral or environmental signal matching [200.3].

Component [200.4] represents optional AI-assisted validation, which may evaluate contextual data, risk profiles, or cross-reference real-time signals. Component [200.5] includes third-party verifiers or external validation modules which may confirm digital identity, payment authorization, or licensing credentials.

Once validation is successful, the system triggers Mint Unit creation via [200.6], the Mint Trigger Interface. This trigger acts as the transitional gate from passive input to active logic, ensuring that no Mint Unit is created without successful confirmation. Failed validations are rejected, logged, and optionally routed for further analysis or reattempt.

This layered, flexible structure allows for complex authentication stacks or simplified flows depending on application requirements.

[300] Mint Unit Generation

Once validation is confirmed, the Mint Unit Generation module [300], shown in FIG. 4, is activated. This component creates a unique Mint Unit—a digital logic object containing metadata used for downstream execution. Mint Units are single-use and may exist on-chain, off-chain, or within isolated system memory depending on application needs.

Subcomponent [300.1] generates a unique identifier (UID) or hash for the Mint Unit, while [300.2] timestamps its creation. Metadata encoding [300.3] embeds the results of the validation, including relevant user data, contextual parameters, and routing logic directives. Optionally, [300.4] applies state or expiration logic, limiting how long the Mint Unit™ is valid for execution.

Component [300.5] packages the Mint Unit in a secured wrapper, optionally cryptographically signed, to prevent tampering. The completed Mint Unit is then passed to the Routing Decision Layer [400] for directional logic evaluation.

[400] Routing Decision Layer

The Routing Decision Layer [400], depicted in FIG. 5, evaluates the metadata within the Mint Unit and determines which execution protocol it should follow. The embedded routing directive may point the Mint Unit to one of four primary logic paths: Mint-to-Burn [500a], Mint-to-Limbo [500b], Mint-to-Access [500c], or Mint-to-Code [500d].

Component [400.1] analyzes the Mint Unit's internal logic and validation context. It cross-references this with a rule engine [400.2], which may include static rules, AI-based logic trees, or user-defined flows. Component [400.3] generates the routing signal, which is embedded in the Mint Unit and used to direct it to the proper execution protocol. The routing decision is then logged for traceability and audit purposes.

This layer enables programmable, modular decision-making that adapts to different applications, such as secure facility access, NFT minting, AI logic deployment, or event ticketing.

[500a] Mint-to-Burn Protocol

The Mint-to-Burn Protocol [500a], illustrated in FIG. 6, is designed for use cases where the Mint Unit is intended to exist only briefly and is destroyed immediately after a single authorized use. This logic path is optimal for applications such as one-time access codes, single-use voting credentials, or expiring transaction keys.

Upon arrival at [500a.1], the Mint Unit is verified for structural integrity. Component [500a.2] confirms the protocol directive and ensures that it matches the routing instruction embedded at the routing layer [400.3]. Execution logic [500a.3] processes the Mint Unit as a one-time credential, initiating its function (e.g., granting access, logging vote, delivering content). Once execution is complete, the Mint Unit is passed to the burn operation [500a.4], which irreversibly invalidates or deletes it.

The final state [500a.5] logs this destruction in the audit ledger, ensuring full traceability without storing personal information (non-PII).

This protocol ensures maximum security, minimal resource retention, and strict one-time-use logic.

[500b] Mint-to-Limbo Protocol

The Mint-to-Limbo Protocol [500b], depicted in FIG. 7, provides a controlled state in which a Mint Unit is held temporarily under conditional access logic. This is useful for situations where AI must make a decision, additional user input is pending, or a time-based condition must be met.

When the Mint Unit enters [500b.1], it is assigned a limbo status and securely stored. Component [500b.2] monitors for conditional triggers such as a secondary verification, AI decision logic [500b.3], or time-delay [500b.4]. If no action is taken within a defined window, the system automatically invalidates the Mint Unit via [500b.5]. If conditions are satisfied, the Mint Unit is either rerouted to another protocol or allowed to proceed.

Optional return loops and recalibration pathways may be integrated here. All states, outcomes, and expiry events are recorded in [500b.6], the audit logger.

This protocol introduces flexibility, decision-making latency, and programmable pauses to enhance systemic integrity.

[500c] Mint-to-Access Protocol

The Mint-to-Access Protocol [500c], shown in FIG. 8, governs access to a physical, digital, or system-level asset upon successful validation and routing. It is commonly used in door locks, system login gates, content portals, or limited access zones.

Upon receiving a Mint Unit at [500c.1], the system runs a final access validator [500c.2] to ensure that conditions (e.g., time, location, identity) still align. Once cleared, access is granted via [500c.3]. Simultaneously, the system may enforce constraints such as time limits [500c.4] or resource scope, and it will initiate an access log through [500c.5].

After access is complete or time has expired, the Mint Unit is destroyed or archived through [500c.6].

This protocol guarantees secure, conditional entry access with immutable auditing.

[500d] Mint-to-Code Protocol

The Mint-to-Code Protocol [500d], as described in FIG. 9, enables the Mint Unit to act as an executable logic object, triggering internal or external code based on embedded instructions.

Once routed to [500d.1], the Mint Unit passes through a logic validator [500d.2] to confirm directive integrity and authorization. If approved, it enters the logic engagement engine [500d.3], which interprets and prepares the Mint Unit for execution. The command is passed to [500d.4], an execution module that may run system code, AI tasks, smart contracts, or webhook commands.

Results of this execution are captured by [500d.5] and logged for feedback, traceability, or future routing. The final state of the Mint Unit is handled in [500d.6], which finalizes or destroys it.

This protocol enables programmable trust, AI task control, and system automation from a secure tokenized entry point.

[600] Audit Trail and Logging

Component [600], referenced throughout the system, is responsible for recording every stage of the Mint Unit lifecycle in a secure, immutable manner. The audit trail preserves non-PII metadata including timestamps, session IDs, validation results, routing decisions, execution outcomes, and destruction confirmations.

Each Mint Unit event generates a corresponding log entry via [600.1], which includes a hash or UID, lifecycle state, protocol path taken, and any conditional metadata. Logs are stored in an encrypted or blockchain-compatible ledger via [600.2] and may be accessed for auditing, security review, or machine learning optimization.

The audit trail subsystem may also include alert triggers [600.3] for abnormal behavior, such as repeated access attempts, protocol re-entry, or tampering signals. The system ensures transparency without compromising user privacy, and it forms the accountability backbone of the Mint-to Logic infrastructure.

This module is optional but strongly recommended for regulated applications such as identity systems, digital voting, financial access controls, or AI transparency layers.

6. Informal Claims

1. A modular logic-based digital execution system, herein referred to as the Mint-to Logic™ system, comprising:

- (a) a validation subsystem configured to receive one or more data inputs from source input channels;
- (b) a trigger mechanism that activates upon successful validation;
- (c) a minting engine that creates a single-use logic object referred to as a Mint Unit;
- (d) a routing layer configured to evaluate the metadata within the Mint Unit and direct it to one or more protocol paths;

(e) at least one protocol module selected from a group consisting of: Mint-to-Burn, Mint-to-Limbo, Mint-to-Access, and Mint-to-Code;

(f) an audit subsystem for storing state, execution, and lifecycle metadata in an immutable or encrypted format.

2. The system of claim 1, wherein the validation subsystem supports biometric, environmental, contextual, password, device, or third-party verification methods.

3. The system of claim 1, wherein the Mint Unit includes a unique identifier, timestamp, validation metadata, and embedded routing logic.

4. The system of claim 1, wherein the Mint-to-Code protocol executes logic instructions embedded in or referenced by the Mint Unit, triggering smart contracts, system automation, or AI task execution.

5. The system of claim 1, wherein the Mint-to-Limbo protocol temporarily stores the Mint Unit in a conditional hold state pending additional input, AI analysis, or time-based events.

6. The system of claim 1, wherein the Mint-to-Burn protocol destroys the Mint Unit upon a single successful use and records the destruction in the audit trail.

7. The system of claim 1, wherein the Mint-to-Access protocol validates permission and grants conditional or time-based access to a secure resource or system state.

8. The system of claim 1, wherein each protocol path logs the execution result in the audit subsystem with non-personally identifiable metadata, session identifiers, and lifecycle state outcomes.

9. The system of claim 1, wherein Mint Units are generated on-chain, off-chain, or within isolated system environments depending on application context.

10. The system of claim 1, wherein the routing layer is dynamic and uses rule-based, AI-driven, or behaviorally adaptive logic to determine execution protocol.

7. Inventor Declaration and IP Disclaimer

I, Spencer Southern, am the sole inventor and developer of the system, components, and logic structures described in this application, including all figures, architectural frameworks, terminology, and associated use cases. I retain 100% ownership of all intellectual property related to this invention and its current or future modular applications, as outlined or referenced herein.

This document represents an original work under U.S. patent law and is being filed as a provisional patent application to secure priority rights.

All content herein is original intellectual property under development by the inventor. Unauthorized reproduction, use, or dissemination is prohibited. Patent(s) pending.

This system, known as the Mint-to Logic framework and including but not limited to the Mint Unit, associated protocols, and validation-to-routing architecture, is protected under provisional status from the date of official filing.

**Spencer Southern
Founder, Southern Star Pro. Studios, L.L.C.
Delton, Michigan
Date: 4/15/2025**